

AI IMPLEMENTATION PROMPT — Kohler catalog + layout + 3D wiring

Paste this file to any coding AI working on this repo. It is the single source of truth for this task. Human manual: docs/MANUAL_GUIDE.md . CSV template: supabase/seed/products_seed.csv .

CONTEXT

Repo: Next.js 16 + React 19 + TS + Tailwind v4 + Supabase (/Users/devraj/Desktop/KOHLER). Existing, do not rebuild:

- src/lib/products.ts — Product type + PLACEHOLDER_PRODUCTS (6 PENDING-* rows, price/dimensions = null).
- src/app/planner/page.tsx — 2D SVG planner, hardcoded PALETTE + overlaps() + undo. Deterministic, no AI.
- src/app/design/new , src/app/budget , src/app/catalog , src/app/saved — Phase 1 UI shell.
- supabase/migrations/20260916000000_phase1_schema.sql — tables products(sku,name,category,collection,region,finish,dimensions,price_inr,official_url,image_urls,source,verified_at) , design_projects , design_versions(layout_json) , design_items .RLS on.
- Design tokens: design.md (void #000 , cream #f5f5f0 , charcoal #202020 , pill 9999px , card 10px).
- Human has hand-filled supabase/seed/products_seed.csv (17 cols, real Studio Kohler data, GLB paths). That CSV is the ONLY product source.

Current gaps: products lacks numeric dims, style tags, asset URLs; category CHECK excludes Vanities ; no Storage buckets; no /api/recommend ; planner not DB-driven; no 3D viewer.

OBJECTIVE

Accommodate the human's CSV catalog so the app can: (1) import + serve real products, (2) AI-recommend a budget- and space-valid bundle with style matching, (3) auto-layout in 2D, (4) preview in 3D from GLBs with 2D fallback.

REQUIREMENTS

1. **DB migration** supabase/migrations/XXXX_catalog_assets.sql :

```
alter table products add column if not exists width_mm int, add column if not exists depth_mm int, add column if not exists height_mm int, add column if not exists style_tags text[] default '{}', add column if not exists model_glb_url text, add column if not exists footprint_svg text, add column if not exists dwg_url text;
alter table products drop constraint if exists products_category_check;
alter table products add constraint products_category_check check (category in ('Showers','Bathtubs','Basins','Smart Toilets','Faucets','Mirrors','Vanities'));
```

2. **Storage**: create public-read buckets models, images, cad (migration or dashboard SQL). Products readable by all (existing policy stays).
3. **CSV import**: script or one-off route reading supabase/seed/products_seed.csv → upsert to products . Mapping: blank numeric → NULL; style_tags split on | ; image_urls split on , ; footprint_svg stored verbatim; verified_at date or now. Log skipped rows with reason. Re-runnable.
4. **Types**: extend Product in src/lib/products.ts with width_mm,depth_mm,height_mm: number|null; style_tags: string[]; model_glb_url,footprint_svg,dwg_url: string|null; image_urls: string[] . Keep fmtPrice (null → "Unknown"). Keep placeholder rows compiling.
5. **Recommend API** src/app/api/recommend/route.ts (POST {room:{l_m,w_m,h_m}, budget_inr, style, prefs}):
- Load products from Supabase (fallback CSV). Pre-filter in TS: price <= remaining (null price = allow but warn), width/depth fits room , style_tags n requested (if zero hits, relax style, flag it).
 - Single LLM call (Vercel AI SDK + Gemini Flash or gpt-4o-mini , env key only, never client-exposed) with: constraints + <40 filtered products {sku,name,category,price,width,depth,styles} + instruction "return ONLY SKUs from list, one per category, total ≤ budget". Enforce strict JSON via zod: {bundle:[{sku,reason}], totalCost, layout:[{sku,x_m,y_m,rotation_deg}], explanation, warnings} .
 - Deterministic re-validation in TS: SKU<catalog, sums≤budget, footprint fits room, no overlap (reuse overlaps logic in meters). On fail: drop offending item + retry once, else return 422 with reason. Never invent SKUs/prices.
6. **Planner wiring**: src/app/planner/page.tsx accepts layout prop / query — replace hardcoded PALETTE dims with DB width_mm/depth_mm (mm→px), draw footprint_svg where present, keep drag/rotate/snap/undo + red clash outline.
7. **3D viewer** src/app/viewer/page.tsx (new, placeholder-safe): three + @react-three/fiber + @react-three/drei , <Canvas> maps layout_json → GLB at (x_m, y_m) or fallback box if model_glb_url IS NULL . Room shell from design_projects dims. No new global CSS theme.
8. **Budget/saved**: /budget sums real price_inr (null → "Unknown" line, excluded from total + warning); save bundle → design_versions(layout_json) + design_items(sku,x_m,y_m,rotation) .

CSV FORMAT (identical to MANUAL_GUIDE — do not change)

```
sku,name,category,collection,finish,price_inr,width_mm,depth_mm,height_mm,style_tags,official_url,image_urls,model_glb_u
K-5401,Veil Wall-Hung Smart Toilet,Smart
Toilets,Veil,White,145000,"700,580,450,minimalist|modern|zen",https://www.studiokohler.com/home/products/veil-k-
```

```
5401,images/K-5401-hero.jpg,models/K-5401.glb,"<svg width=""70"" height=""58"" viewBox=""0 0 70 58""><rect x=""2"" y=""2"" width=""66"" height=""54"" rx=""18"" /></svg>",cad/K-5401-plan.dwg,studiokohler.com,2026-09-17
```

Rules: header exact order; `category` $\in$ 7 above; numerics digits-or-blank $\rightarrow$ NULL; `style_tags` | -split; `footprint_svg` verbatim; `sku` unique; never invent values — blank stays NULL/Unknown.

CONSTRAINTS

- Real data only (plan.txt NON-NEGOTIABLE): no fake SKUs/prices/dims/URLs/compatibility.
- AI for taste/language only; TS for money/geometry/permissions/DB access.
- Secrets server-side only (`process.env`), RLS respected, no service-role key in client.
- Follow `design.md` tokens; Motion default; no new UI theme.
- No deps beyond `ai`, `@ai-sdk/google` (or `openai`), `zod`, `three`, `@react-three/fiber`, `@react-three/drei` unless justified.

EXCLUSIONS (do not touch)

- Landing `/`, `nav/footer`, `auth`, `design/new` form fields (only read from them).
- DWG/OBJ $\rightarrow$ GLB conversion itself (human does it; you only consume `model_glb_url/footprint_svg`).
- `api-dev-portal.kohler.com` integration, scraping, price FX conversion.

ACCEPTANCE CRITERIA

- ☐ Migration applies clean on fresh Supabase; `products` holds CSV rows with NULLs preserved.
- ☐ `POST /api/recommend` with `{room 3.6×2.4m, budget, style Zen}` returns bundle 100% from CSV, total $\leq$ budget, footprints fit, no overlaps; unknown-price items flagged.
- ☐ Planner renders AI `layout` with DB dims + SVG footprints; drag/rotate/undo + clash warnings still work.
- ☐ `/viewer` shows GLBs where present, boxes elsewhere; never crashes on missing assets.
- ☐ `npm run build + tsc --noEmit` clean; no secrets in client bundle.

VERIFICATION

```
npm run build
npx tsc --noEmit
# seed check
psql $DATABASE_URL -c "select category,count(*),count(model_glb_url) from products group by 1;"
curl -X POST localhost:3000/api/recommend -H 'Content-Type: application/json' -d '{"room":
{"l_m":3.6,"w_m":2.4,"h_m":2.7},"budget_inr":300000,"style":"zen","prefs":"freestanding tub if fits"}'
```

Expect HTTP 200, SKUs $\subseteq$ CSV, `totalCost` $\leq$ 300000, `layout` inside room bounds. Test style-miss (request `zen` with classic-only catalog $\rightarrow$ bundle returned + `warnings` notes relaxation) and missing-GLB SKU (2D OK, 3D box fallback).